



Unit 8 Quiz (Friday November 14) [Click for Study Guide]

Completion requirements

Due: Sunday, December 14, 2025, 10:20 AM

DSC 360: Review Guide for Unit 8: From Prototype to Production

This will be our **last quiz**. It covers everything in the sessions:

- “RAG Revisited: Grounding Answers with Evidence” (11/7)
- “Ship It Safely: Choosing and Justifying Your LLM Deployment” (11/10)

Unit 8 brings together the two sides of building an AI-powered application: *getting the answers right* and *delivering them safely*. We revisit RAG to ensure outputs are grounded in evidence, then shift from classroom demos to real deployment—choosing where your model runs, adding moderation and safety gates, and planning for latency, cost, and reliability.

Terms to Know

Be ready with a working definition or use in a sentence.

RAG Revisited

- **RAG:** Retrieve evidence, then generate **only** within it (or abstain).
- **Dense retrieval:** Semantic vectors/KNN; great for paraphrase and “what it’s about.”
- **BM25 (sparse):** Keyword/ID matching; great for exact tokens, codes, names.
- **Hybrid retrieval:** Combine dense + BM25; often best-of-both.
- **Similarity cutoff:** Below threshold → **abstain** or ask for clarification.
- **Evidence-only answering:** Use provided passages/rows and **show** them.
- **Faithfulness (groundedness):** Every claim supported by retrieved evidence.
- **Hit@K:** Did a correct item appear in the top K retrieved?
- **Re-ranking:** Second-stage scorer to push the best evidence to the top.
- **Extraction-first:** Convert messy text → structured JSON/table; retrieve over that.
- **SQL/API tool use:** Read-only facts/aggregations from a database or service; “show your work.”
- **Agent loop:** Perceive → Decide (pick tool) → Act → Reflect; tiny policy around RAG.

LLM Deployment

- **Endpoint:** A URL for one job (e.g., `/chat`, `/embeddings`, `/moderations`). Same app, different door.
- **Moderation gate:** Scan **all untrusted input/output**; if flagged → refuse and log (fail-closed).
- **Fail-closed:** On any safety flag, **do not** generate or deliver; return a safe message.
- **Local / Managed / Hybrid:** Self-hosted; provider-hosted; or a mix (e.g., local pre/post + hosted model).
- **p50 / p95 latency:** Typical vs worst-case response time targets you set.
- **Throughput:** Requests per minute you need at peak.
- **Rate limit (HTTP 429):** You hit provider quotas; pause and retry.
- **Timeout:** No reply in time; retry once or twice, then fail fast.
- **Exponential backoff:** Wait longer on each retry (1s → 2s → 4s...). Cap at ~2-3 retries.
- **Circuit breaker:** Stop calling a failing service; cool down; probe; resume when healthy.
- **Canary rollout:** Send a **small** % of traffic to a new version first; auto-rollback on bad metrics.
- **Version pinning:** Lock the model/config so it doesn't change silently.
- **Provider adapter (swap):** Tiny layer that maps your calls to different providers/models.
- **Kill switch:** One toggle to pause generation and return a safe fallback message (e.g., service paused).
- **Read-only DB role / allow-listed tools:** Least-privilege access; restrict what the app can do.
- **PII redaction:** Remove personal identifiers before logging or sending.
- **Staging vs prod keys:** Test risky prompts in **staging**, never with production keys.

Short Answer and Problem Solving

Be prepared to discuss...

- **Why retrieve at all?** Ground answers, fight drift, and fit more than the context window.
- **Safety pipeline (exact order):** pre-screen → moderation → generation → output check; fail-closed on any flag.
- **Guardrails in practice:** similarity cutoff + abstain; evidence-only answers; read-only DB role; allow-listed tools/domains; minimal-PII logging/redaction.

- **Deployment choice by scenario:** when local makes sense (privacy/control, small scale), when managed wins (spiky traffic, no ops), and why **hybrid** is common (local pre/post + hosted model).
- **Pre-screening latency:** how to avoid it being the bottleneck (cheap local checks, short prompts, cache results, only send redacted text to vendor moderation).
- **Cost/latency trade-offs:** do napkin math (tokens → \$) and set a p95 target; how to lower latency (cache, trim context, smaller models).

Make sure you understand the difference between...

- **Dense vs. BM25 vs. SQL/API:** paraphrase meaning vs exact tokens vs factual queries/aggregations.
- **Open-weights vs closed-source and self-hosted vs managed:** model licensing and hosting are **separate axes** (open models can be managed/hosted).
- **Rate limit vs timeout:** quota exceeded vs no timely reply.
- **Exponential backoff vs queueing:** wait longer between retries vs hold requests to smooth bursts.
- **Circuit breaker vs “just retry”:** stop hammering a failing service, cool down, probe, then resume—don’t keep escalating retries.
- **Canary rollout vs A/B test:** safety rollout to a small % with auto-rollback vs experiment to compare variants.

Be ready to give one tiny example...

- **Exponential backoff:** “If a call fails: wait 1s, then 2s, then 4s (cap at N tries); reduces load and respects quotas.”
- **Cost napkin math:** “2k in + 500 out at \$1M/\$2M ≈ \$0.003 per call (=\$2 + \$1 per 1,000 calls).”
- **Fail-closed response:** “Flagged content → refuse safely and log; do **not** generate.”
- **Safety pipeline order:** “Pre-screen → Moderation → Generation → Output check (fail-closed).”

Example Essay Prompts (for Paper Test 2)

From RAG to Agents

A teammate says: “RAG was fine for prototyping, but in production we need an agent. RAG is just KNN over a vector database.” Explain what this misses. Define what RAG actually is, why we retrieve in the first place, and how a simple agent loop often **wraps** RAG rather than replaces it. Use course vocabulary and one concrete example from class.

Strong essays will address: why we retrieve to ground answers and handle drift/context limits; how retrieval extends beyond vectors (e.g., BM25, SQL/API, extraction-first) and when that matters; where RAG fits inside a minimal perceive–decide–act–reflect loop; and one concrete production safeguard or metric with a brief recommendation for the team.

Ship It Safely: Deploying Applications at Scale

You're turning a small Course Registration RAG-based helper into a real service for add/drop at a small liberal arts college. With a tight budget and some PII in advising notes, explain how you would deploy it so it stays safe, reliable, and affordable.

Strong essays will address: hosting choice (local/managed/hybrid) with one or two key trade-offs; a moderation-first, fail-closed safety pipeline (and minimal-PII logging); one reliability control you'd use (e.g., backoff, circuit breaker, canary/rollback) and when it triggers; a simple measurement/go-no-go plan (e.g., p95 latency target, daily cost, tiny gold set); and how you'd handle provider change (pin versions, staging vs. prod, swap path). *Note from Prof. Allen: My concern is how to handle latency for pre-screening. We don't want that to be our bottleneck. You need to convince me that you have a practical grasp of this, not just make up numbers.*